

On the Quest for Energy Efficient UDP Sockets

Rui Pedro C. Monteiro
University of Minho &
INESC TEC
rui.p.monteiro@inesctec.pt

João Marco C. Silva
University of Minho &
INESC TEC
joaomarco@di.uminho.pt

Abstract—The increasing share of User Datagram Protocol (UDP) traffic on the Internet, driven by latency-sensitive applications and the rise of protocols such as Quick UDP Internet Connections (QUIC), coincides with growing concerns over the energy footprint of network infrastructures. When handling UDP traffic, developers can choose among several socket programming strategies, ranging from the standard *AF_INET* Application Programming Interface (API) to kernel-bypass alternatives such as *AF_XDP* sockets. This paper investigates whether adopting a more performant socket API, even when conventional ones already keep up with demand, can reduce power consumption. Using a controlled experimental setup, we measured the power consumption, Central Processing Unit (CPU) utilization, and packet loss of blocking, non-blocking, and batch configurations across both socket families, at rates up to 1.2M packets per second. Blocking *AF_XDP* sockets emerged as the most energy-efficient option under medium-to-high demand, saving up to 33% of energy while sustaining zero packet loss. A real-world evaluation with the NLnet Labs Name Server Daemon (NSD) Domain Name System (DNS) server confirms these gains.

Index Terms—UDP, *AF_XDP*, XDP, eBPF, energy efficiency, network programming, sockets, sustainability

I. INTRODUCTION

For a long time, Transmission Control Protocol (TCP) has dominated the Internet as the most widely used network transport-level protocol. In recent years, this trend has begun to shift with the emergence of latency-sensitive applications such as Web Real-Time Communication (WebRTC) and online multiplayer games. This trend, together with the emergence and rise of new protocols like Quick UDP Internet Connections (QUIC), leads to a consequent increase in User Datagram Protocol (UDP) network traffic across the Internet [1].

Simultaneously, two other subjects have gained popularity in recent years. A growing concern about the Information and Communication Technology (ICT) sector’s environmental impact [2], [3], and therefore, network infrastructures, has sparked numerous research efforts in the search for energy-efficient strategies. On the other hand, the continuously increasing demand provided the development of new high-performance network processing approaches.

Aside from technological developments and infrastructure improvements, the responsibility for achieving better sustainability falls on all involved in an application’s lifecycle. Writing efficient code often reduces computational load, which in turn reduces power consumption. Developers typically pursue code optimization not due to energy concerns but rather out

of necessity when faced with bottlenecks and limitations that impact an application’s Quality of Service (QoS).

When building applications and addressing low-level network programming, the same logic applies, and the choice often relies on the most practical underlying Application Programming Interface (API) available. Most UDP-based applications use the standard socket’s API provided by the Operating System (OS) as the default option, which, thanks to modern hardware capabilities, is often more than capable of handling the imposed network demand. Recently, more performant and therefore more efficient APIs have become available. However, these are usually considered a last resort in extreme cases of network demand.

This work seeks to answer whether the use of more efficient and performant socket APIs, even when traditional ones can keep up with demand, can reduce power consumption and, consequently, environmental impact. We study the power consumption, Central Processing Unit (CPU) usage, and packet loss percentage behind different network programming approaches available to developers when designing UDP-based applications. In this way, its main objective is to provide a comprehensive analysis of the overall power consumption and performance implications of distinct alternatives for handling UDP traffic.

This document is organized as follows. Section II presents an overview of the current different UDP alternatives. Section III details both the experimental setup and the test scenarios used to assess each programming strategy in terms of energy efficiency. Section IV presents and analyzes the experimental results regarding power consumption, CPU utilization, and packet loss. Section V validates these findings on a real-world application, *i.e.*, the NLnet Labs Name Server Daemon (NSD) Domain Name System (DNS) server. Section VI discusses the main implications of the obtained results, and Section VII concludes the paper.

II. BACKGROUND

When addressing UDP traffic in low-level programming, there are a few *socket families* and configuration options available. The most common socket family on Linux is *AF_INET*, which specifies the usage of Internet Protocol version 4 (IPv4). *AF_INET6* similarly specifies the usage of Internet Protocol version 6 (IPv6). However, its usage is outside the scope of this work.

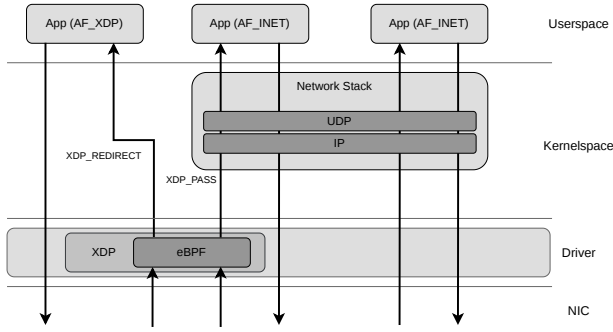


Fig. 1. AF_XDP vs AF_INET

Together with *AF_INET*, the *SOCK_DGRAM* option specifies a datagram-based protocol, which, by default, corresponds to UDP. When instantiating a socket using these options, each received packet will traverse the Linux network Kernel, which will in turn handle any header parsing before finally delivering the packet to the application layer.

Additionally, these sockets can be operated in a blocking or non-blocking mode. In the first mode, when the application calls a function to read a new packet (e.g., *recvfrom*), if no packets are available, execution will block until the OS sends an interrupt when new packets become available. In the second mode, instead of blocking when no packets are available, execution proceeds, allowing a busy-waiting cycle that constantly checks for new packet arrivals.

Regardless of blocking or non-blocking mode, the application can receive multiple packets at a time when the Kernel receives multiple packets in a short period. This approach effectively allows batching the processing of multiple packets at once and reduces the number of calls to *receive family* functions.

Another, more recently available socket family is *AF_XDP*, often referred to as *AF_XDP* Sockets (XSK). This strategy leverages extended Berkeley Packet Filters (eBPFs) programs, more precisely eXpress Data Path (XDP), to bypass the entire Kernel's network stack and deliver raw packets directly to the application in user space [4]. Similar to *AF_INET*, these sockets can also be operated in blocking or non-blocking modes. Although possible to receive a single packet at a time, this strategy is designed in such a way that batch processing becomes the most appropriate usage method.

Figure 1 depicts a packet's path when using *AF_INET* sockets or XSK, highlighting the main differences between the two socket families. When using solely conventional *AF_INET* sockets, upon receiving a network packet, the Network Interface Card (NIC)'s driver will call the *alloc_skb()* function to allocate the required network buffer. The packet is later handed to the Kernel's network stack, where it will, for example, traverse the Internet Protocol (IP) and UDP layers.

XSK makes use of an XDP program to bypass the previously described flow. These eBPF programs act as an early hook that intercepts a packet and can modify it and determine

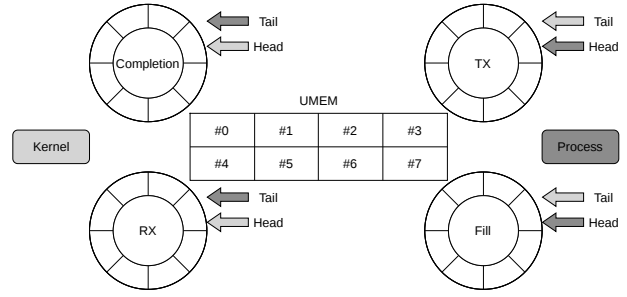


Fig. 2. XSK Rings

its fate before the *alloc_skb()* call. After the interception, there is a fixed set of actions that an XDP program can apply to a packet.

The *XDP_DROP* action determines a packet to be dropped, a particularly useful function in mitigating Denial of Service (DoS) attacks, since the packet is discarded early. *XDP_ABORTED* is used to indicate that an unrecoverable error has occurred, halting the packet processing. *XDP_TX* bounces the packet back through the same interface it arrived on. Lastly, *XDP_PASS* allows the packet to continue through the Kernel's stack, while *XDP_REDIRECT* redirects the packet to a few possible locations.

A packet can, for instance, be redirected to another interface; however, the ability to redirect it straight to *user space* enables XSK. The XDP program acts as a filter, steering application traffic directly to *user space* while allowing the remaining network packets to follow their natural path through the Kernel's stack.

As shown in Figure 2, besides the XDP packet redirecting mechanism, XSK works thanks to a piece of memory shared between *kernel space* and *user space*, called *UMEM*, and four distinct *ring buffers* (*TX*, *RX*, *Completion*, *Fill*) [5]. The *UMEM* behaves like an array, divided into individual *chunks* that contain packet data. In turn, the four ring buffers are used to transfer ownership of each chunk between different spaces without requiring synchronization techniques, e.g., locks.

Each ring has a single producer and a single consumer. The Kernel acts as a producer to the *RX* and *Completion* rings and as a consumer to the *TX* and *Fill* rings, while the *user space* application fills the opposing roles [6].

The *RX* and *Fill* rings are used to handle the receiving path, while the *TX* and *Completion* rings concern the transmitting path. Upon start-up, the application owns all the *UMEM* chunks. To receive packets, the application needs to transfer chunks to the Kernel via the *Fill* ring. The Kernel will then use those chunks to place the packets steered by the XDP program and transfer ownership back through the *RX* ring so the application can access the packet data. Once the application finishes processing those packets, the chunks can be handed again to the Kernel, repeating the cycle.

Sending packets works following a similar process. The

application puts packets into its own available *UMEM* chunks and later transmits ownership of the chunks to the Kernel via the *TX* ring. After completing the transmission of those packets, the Kernel returns the *UMEM* chunks back to the application using the *Completion* ring, allowing the cycle to repeat.

Together, these mechanisms make XSK a highly performant solution for network traffic processing, both for receiving and transmitting operations. By avoiding all the complexity of the kernel's network stack and the consequent memory copies involved, the *AF_XDP* family of sockets has proven itself to be one of the fastest network programming technologies [7].

However, circumventing the Kernel's stack also adds implications. Functionality usually provided by the Kernel is lost, requiring additional work to be done by the application, such as parsing the headers from the raw packet. As another example, TCP applications are much harder to convert to XSK, since the entire TCP logic, like congestion and session management, would need to be implemented in *user space*. Nevertheless, UDP is connectionless. Therefore, most communications that use this transport protocol already implement a large portion of their connection logic in user space (*e.g.*, QUIC), which facilitates the transition.

Besides XSK, other Kernel bypass solutions exist, such as Data Plane Development Kit (DPDK) [8]. Although a well-known alternative that has demonstrated even better performance than XSK in some scenarios [9], we have decided not to include it in this comparison due to its operating details. Contrary to XSK, DPDK can not be opted in on an application basis, since its usage implies a complete hijack of the NIC and disrupts all other applications on the same machine. As such, DPDK is a less transparent choice that is not entirely within the developer's control when designing an application.

Another relevant socket family is *AF_PACKET*, which also delivers raw packets directly to the application. Yet, these sockets operate similarly to a *network tap*, meaning that a clone of all packets received by the NIC, regardless of protocol or port, will be delivered to the socket. This behavior not only implies additional overhead but also the need to filter traffic at the application layer, since it will receive packets not destined to it. As such, we also decided not to consider this option as it is impractical for the development of general-purpose applications.

III. ENERGY EFFICIENCY STUDY

To measure the energy efficiency of both *AF_INET* and *AF_XDP* sockets, an experimental setup and a series of experiments were devised. Both the setup and the tests are designed to recreate a network application handling incoming traffic on a computing server. Simultaneously, their purpose is to assess the QoS impact on the application and the behavior of the overall machine.

A. Experimental Setup

The experimental setup is mainly composed of the Device Under Test (DUT) (*i.e.*, server), equipped with a 6-core Intel

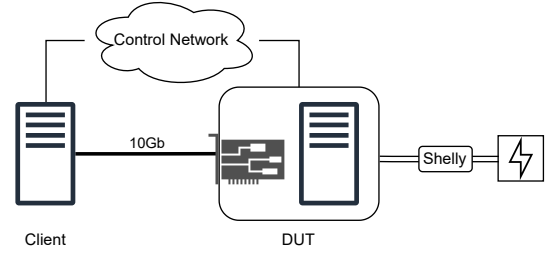


Fig. 3. Experimental Setup

i5-9500 at 3.00 GHz CPU, 16 GB of DDR4 2666 MHz Random Access Memory (RAM), and a consumer-grade Intel 10G X550T NIC, running Ubuntu 20.04 LTS and Kernel version 5.4. As depicted in Figure 3, through its dedicated Intel NIC (experimental interface) and a 10Gb Ethernet cable, the DUT is directly connected to an identical machine that generates network traffic, *i.e.*, the client. Furthermore, it is also connected to a control network over its onboard 1Gb NIC (control interface).

For power consumption measurement purposes, the DUT's power supply is connected to a Shelly PM Mini Gen3¹ that offers real-time energy metrics collected by an *aggregator* device. The setup is located in a controlled-temperature environment that ensures a consistent baseline temperature across all tests. Detailed CPU usage is actively collected using *mpstat*.

The *client* and *server* consist of two applications written in C. The *client* sends a constant packet rate of a fixed packet size to the *server* exclusively over the 10Gb cable. On the other end, the *server* application receives and handles network packets, parsing their payloads to simulate real activity. For the *AF_XDP* family of sockets, since the packets are directly delivered to the application, the *server* also performs, at the software level, checksums for the IP and UDP layers in order to achieve the same functionality as the remaining approaches. Packet loss statistics are computed by comparing the number of packets sent by the client application with the number of packets successfully processed by the server application.

In an effort to standardize the tests and ensure equal grounds between the distinct approaches, a set of configurations was imposed for each socket type. For XSK, the *UMEM* is allocated with space for 4096 frames of size *XSK_UMEM_DEFAULT_FRAME_SIZE* (*i.e.*, 4096 bytes), resulting in a total footprint of 16777216 bytes. Similarly, the UDP receive buffers used by the *AF_INET* socket family are also set to a size of 16777216 bytes, ensuring equal memory requirements.

To guarantee fairness in batch operations, the batch size used with *AF_INET* sockets (blocking and non-blocking) and the maximum number of packets that an *AF_XDP* socket can consume from the *fill ring* at a time were set to the same 512-packet value. Furthermore, all tests of XSK were conducted

¹<https://www.shelly.com/products/shelly-pm-mini-gen3>

TABLE I
TEST CONFIGURATIONS

Test Designation	Packet Rate (pkts/s)	Packet Size Variations (bytes)
Idle	0	N/A
1k	10^3	128, 512, 1024
5k	5×10^3	128, 512, 1024
10k	10×10^3	128, 512, 1024
100k	100×10^3	128, 512, 1024
200k	200×10^3	128, 512, 1024
500k	500×10^3	128, 512, 1024
800k	800×10^3	128, 512, 1024
1M	10^6	128, 512, 1024
1.2M	1.2×10^6	128, 512, 1024

with *zero copy*, ensuring that the NIC directly accesses the *UMEM* regions without the need for the Kernel to execute any additional packet copy.

Additionally, all tests were run with a 2-minute interval between them to prevent any kind of interference and to allow the DUT's temperature to stabilize between runs. For all runs, the receiving *server* application was bound to Core 0.

B. Methodology

The tests conducted aim to assess how XSK and *AF_INET* sockets compare across several configurations. *AF_INET* sockets are evaluated while receiving a single packet at a time and while batching multiple packets at once. As for XSK, tests were conducted in batch mode. Additionally, for each previous scenario, tests were executed in both blocking and non-blocking modes.

To assess how each programming strategy performs, power consumption, CPU utilization, and packet loss were measured across different experimental scenarios over 10 minutes. Table I summarizes each evaluated scenario. The first *idle* scenario recreates a situation in which no traffic is received by the DUT and the *server* application is simply waiting for new packets to arrive. Note that, during the *idling* tests, the DUT still receives some residual network traffic through its control interface. However, this traffic activity is present in all the remaining test cases.

In order to also evaluate each strategy under stress scenarios, tests with constant network traffic demands were conducted at packet rates ranging from 10^3 to 1.2×10^6 packets per second (pkts/s). For each rate, distinct packet sizes were also considered: 128, 512, and 1024 bytes, including headers (see Table I).

We have capped our tests at 1.2×10^6 packets per second, since, at this rate, with a packet size of 1024 bytes including headers, the resulting bit rate is approximately 10Gbps, fully saturating the DUT's experimental interface described in Section III.

IV. EXPERIMENTAL RESULTS

In the following sections, all results presented for power consumption, CPU utilization, and packet loss percentages

are averaged over 10 individual runs, each lasting 10 minutes under constant conditions.

A. Power Consumption

Figure 4 depicts the average instantaneous power consumption of the DUT measured at 1-second intervals, in Watts (W), and for the different test scenarios.

As expected, all three blocking strategies demonstrate significantly lower power consumption when at idle and under low to medium traffic demand scenarios (see Figure 4). At idle, all blocking methods average around 20 W of consumption, with no significant differences among them. In the case of non-blocking approaches, both the standard and batch sockets average approximately 45 W, a 125% increase compared with the blocking approaches. Notably, non-blocking *AF_XDP* sockets show a slightly higher consumption of 3 W in contrast with the other two non-blocking strategies. This gap is noticeable up to around 500k packets per second, where XSK consistently shows a small but noticeable increase in power consumption among the non-blocking approaches.

Takeaway #1. Blocking approaches are more power efficient in lower demand scenarios.

From the combined test results, it is clear that overall power consumption increases with packet rate. However, at higher rates, the gap between blocking and non-blocking strategies shrinks and becomes less noticeable. The exception occurs with XSK Blocking, where the average power consumption appears to hold constant with more than 5k packets per second. Consequently, blocking *AF_XDP* sockets exhibit the best results from a power consumption standpoint for medium to high traffic rates. For instance, with a packet size of 1024 bytes, at a rate of 1.2M packets per second, XSK Blocking averages around 40 W, 28% less than the second-best alternative (XSK non-blocking) and 33% less than common blocking sockets.

Takeaway #2. With an increase in traffic demand, the gap in power consumption between blocking and non-blocking *AF_INET* sockets diminishes.

Takeaway #3. Power consumption from *AF_XDP* sockets remains mostly unaltered independently of the amount of processed traffic.

The results further demonstrate that, once again, except for the XSK Blocking approach, the network packet size does not significantly influence the average power consumption independent of the packet rate. For each approach, power consumption varies solely with the demand rate and shows no noticeable dependence on packet size. For blocking *AF_XDP* sockets, it is, however, possible to observe that an increase in consumption occurs with larger packets. As an example, at 1.2M packets per second, receiving 1024-byte packets has approximately 33% higher power consumption (10 W) than receiving 512-byte packets.

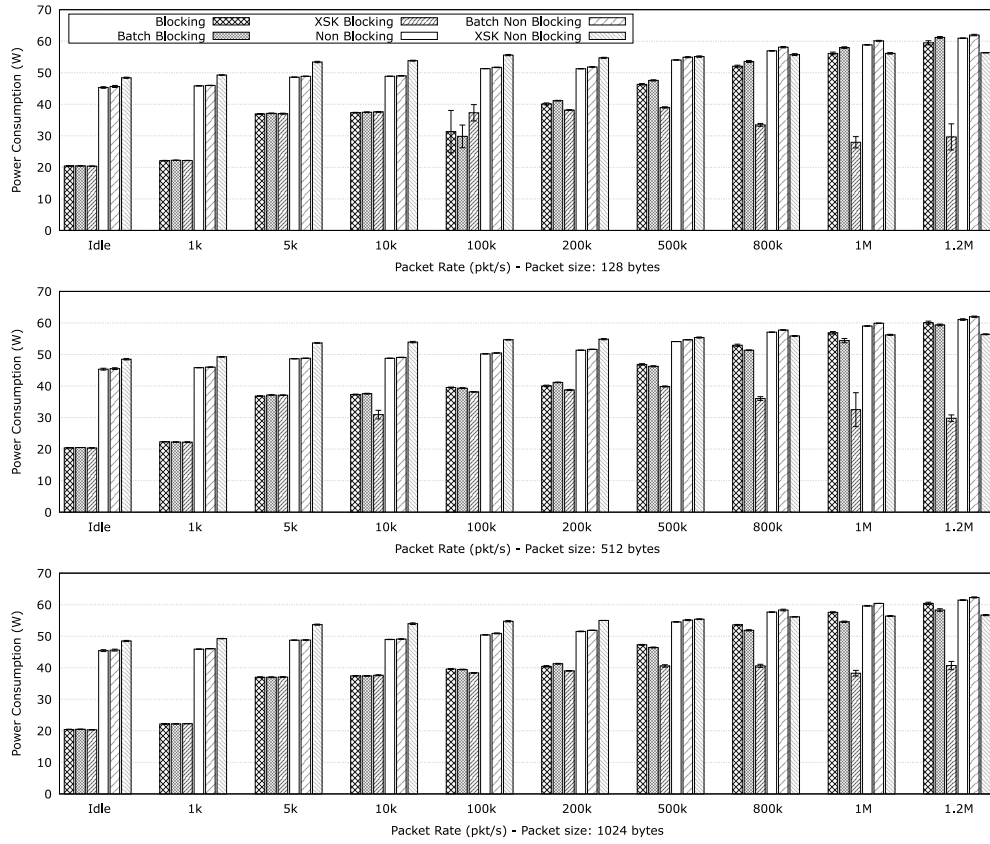


Fig. 4. Average power consumption (W)

Takeaway #4. The size of packets being processed does not influence the power consumption of AF_INET sockets. With XSK, however, a slight increase in power consumption is observed as the packet size grows.

Takeaway #5. At higher packet rates, blocking XSK stands out as the most energy-efficient approach with a significantly lower power consumption.

B. CPU Utilization

The average CPU utilization across all six cores in the DUT, measured at 1-second intervals, is depicted in Figure 5.

With an expected, similar pattern to power consumption, all the blocking strategies exhibit significantly lower average CPU utilization at idle compared with their non-blocking counterparts. Blocking strategies maintain an average of approximately 0.5% CPU utilization across all cores when no traffic is received. Meanwhile, non-blocking approaches, characterized by their active polling, which implies one CPU core at 100% usage at all times, end up averaging around 17.5% utilization overall, a staggering 3400% higher value.

Takeaway #6. Active polling in non-blocking strategies results in a significantly higher CPU utilization compared to blocking approaches.

As the network demand rate increases in the tests, the CPU utilization pattern observed at idle remains relatively constant up to 100k packets per second, with no significant changes as the packet rate increases. Both blocking and non-blocking approaches remain largely tied to their respective models (see Figure 5). From this point onwards, AF_XDP sockets start to distinguish themselves from the alternative strategies, with particular emphasis on the blocking group.

As an example, at a rate of 500k packets per second, XSK blocking results in an average CPU utilization between 1.3% and 2.6%, depending on packet size. Simultaneously, common blocking sockets average around 13% for the same scenarios. As such, XSK sockets yield an 80% to 90% lower CPU usage at 500k packets per second.

Considering the same 500k packets-per-second scenario, within the non-blocking strategies, AF_XDP sockets achieve an average of 17% CPU usage, while the other approaches reach 22%. The most substantial gap between non-blocking XSK sockets and the remaining active-polling strategies, however, is observed at 1.2M packets per second, where the CPU usage drops from approximately 28% to 19% on average, representing a 32% reduction.

Takeaway #7. XSK stands out once more by demonstrating significantly lower CPU usage than the alternatives, particularly in blocking mode.

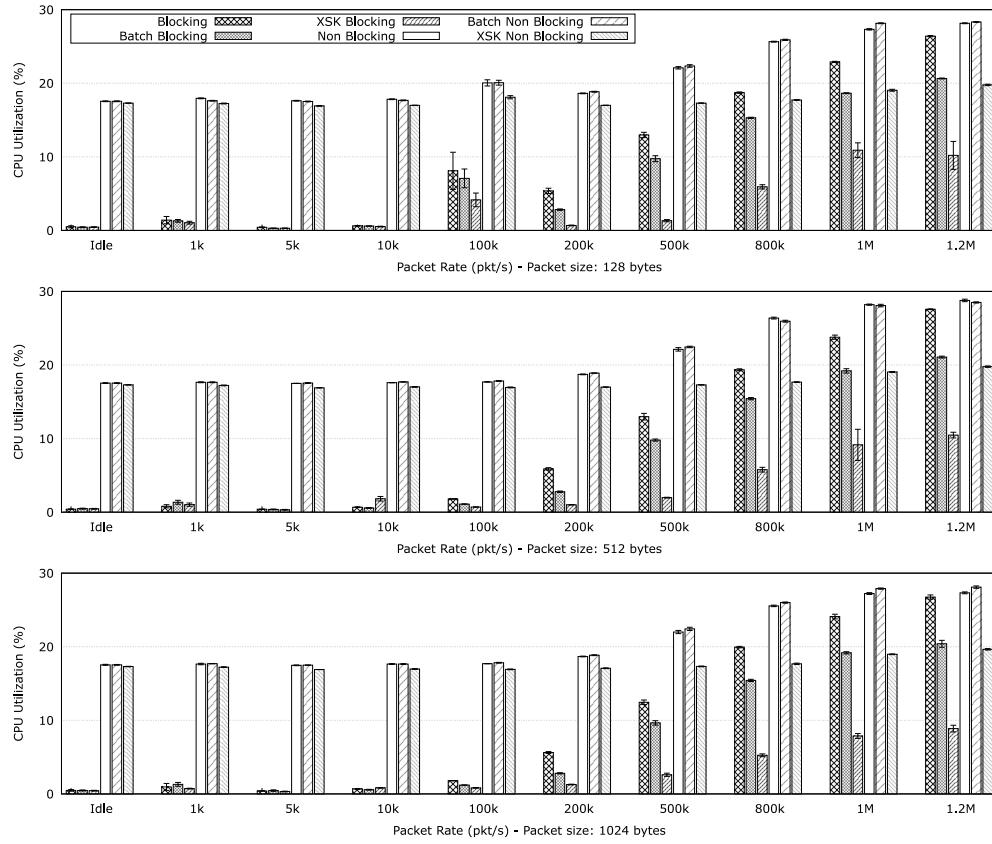


Fig. 5. Average CPU core utilization across all cores (%)

Figures 6 and 7 provide a closer look at the utilization of each of the six individual CPU cores in the DUT when using common blocking sockets and XSK blocking, respectively. Both plots show a 10-minute interval from a single test run during the 1.2M packets-per-second test, using packets of size 1024 bytes. Different scenarios exhibit similar overall behavior; therefore, the same conclusions can be drawn about the detailed usage of each core. The first large noticeable difference between the two approaches occurs in the utilization of Core 0, the one to which the receiving application is pinned. With the standard sockets API, the utilization of this particular core stays close to 100% during the entire test. In contrast, using blocking *AF_XDP* sockets yields just 40%-60% utilization on the same core.

Analyzing the remaining cores, it can be noted that, besides Core 0, only one other core shows significant utilization. In the case of common blocking sockets, this is Core 5 (see Figure 6), while for blocking XSK it is Core 1 (see Figure 7). Although both have a noticeable utilization, in the case of the standard socket API, it is considerably higher, approximately 60% utilization, when compared with the *AF_XDP* alternative that averages just 20%.

Takeaway #8. *The usage of XSK reduces the load on each individual CPU core required to handle the network demand.*

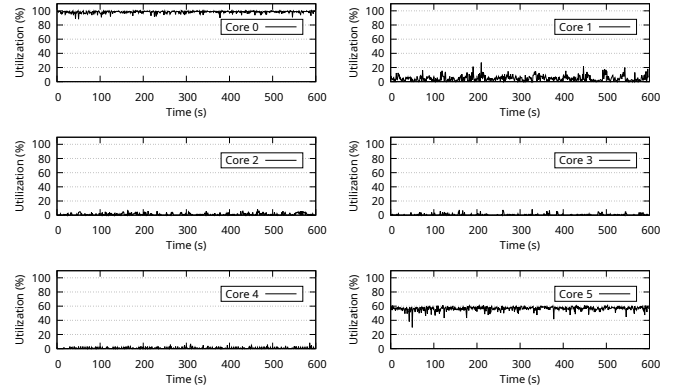


Fig. 6. CPU utilization in *AF_INET* Blocking (1.2M pkts/s of size 1024 bytes)

C. Packet Loss

Table II depicts the average packet loss percentage measured across the different test scenarios. Since no losses occur in the link between the client and server applications, it can be concluded that any packet drops measured are caused by the DUT's inability to handle the network traffic demand.

Regardless of socket type or packet size, no significant packet loss was observed at packet rates ranging from 1k to

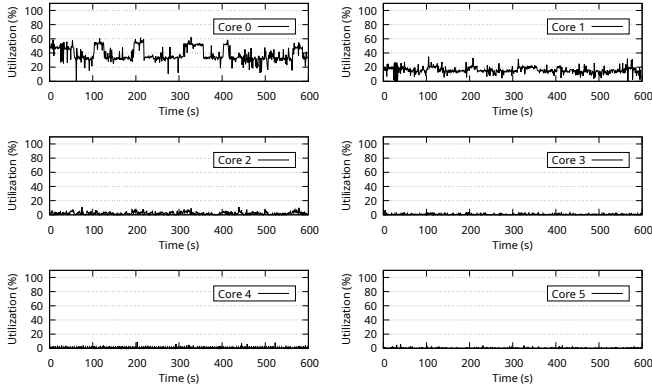


Fig. 7. CPU utilization in XSK Blocking (1.2M pkts/s of size 1024 bytes)

TABLE II
PACKET LOSS PERCENTAGE ACROSS DIFFERENT SOCKET TYPES AND MESSAGE RATES FOR 128, 512, AND 1024 BYTE PAYLOADS.

Packet Size (Bytes)	Socket Type	Packet Loss (%)			
		1k-500k	800k	1M	1.2M
128	Blocking	-	0.01	0.01	0.04
	Batch Blocking	-	-	-	-
	XSK Blocking	-	-	-	-
	Non-blocking	-	-	-	0.04
	Batch non-blocking	-	-	-	-
	XSK non-blocking	-	-	-	-
512	Blocking	-	0.01	0.01	0.15
	Batch Blocking	-	-	-	-
	XSK Blocking	-	-	-	-
	Non-blocking	-	-	0.01	0.22
	Batch non-blocking	-	-	-	-
	XSK non-blocking	-	-	-	-
1024	Blocking	-	0.01	0.01	1.10
	Batch Blocking	-	-	-	0.07
	XSK Blocking	-	-	-	-
	Non-blocking	-	-	0.01	1.32
	Batch non-blocking	-	-	-	-
	XSK non-blocking	-	-	-	-

500k. The first signs of relevant packet drops occur at 800k packets per second, where the standard sockets API starts to show approximately 0.01% of packet loss across all packet sizes. For this particular socket, the trend continues at 1M packets per second, with about 0.01% packet loss across all variations. At 1.2M packets per second, however, it is possible to see even higher drop rates: 0.04% for 128-byte packets, 0.15% for 512-byte packets, and 1.1% for 1024-byte packets.

Takeaway #9. Up to 500k packets per second, all the evaluated network programming strategies manage to handle the demand without any significant packet loss recorded.

The non-blocking counterpart of the common sockets' API performs better at 800k packets per second by demonstrating no packet loss at this rate. However, at 1M packets per second, it follows a similar pattern to the blocking alternative, dropping

around 0.01% of packets of sizes 512 and 1024 bytes. At 1.2M packets per second, for packets of size 128 bytes, it performs identically to the blocking version, but for packets of size 512 and 1024 bytes, it performs surprisingly worse. It shows 0.22% packet loss for 512-byte packets and 1.32% for 1024-byte packets.

In the case of batching strategies, these yield significant improvements by reducing packet loss to zero across all test scenarios, except for packets of size 1024 bytes at 1.2M packets per second. Here, a 0.07% packet loss is measured. However, this result still outperforms simple blocking sockets by 36%.

Takeaway #10. Batch handling network packets improves the results achieved by AF_INET sockets.

As for AF_XDP sockets, both blocking and non-blocking versions sustain zero packet drops across all test scenarios, beating all remaining strategies.

Takeaway #11. XSK achieves perfect results with minimal packet loss across all test scenarios.

V. REAL APPLICATION ANALYSIS

To better understand how the results presented in Section IV correlate to real-world applications, in this section, we measure the implications behind the utilization of AF_INET and AF_XDP sockets in a real scenario. The analysis is performed using the NSD², an authoritative and open-source DNS server designed for raw performance.

DNS is a protocol that commonly operates over UDP, as such, this application represents a realistic test case scenario. By default, NSD uses blocking AF_INET sockets in batch mode to maximize performance. Yet, it also provides out-of-the-box support for blocking AF_XDP sockets via a compile-time configuration option.

As a real application, NSD handles both incoming (requests) and outgoing (responses) traffic. Thus, unlike the receive-only synthetic workload evaluated in Section IV, this scenario also exercises the transmitting path of each socket family, providing a more complete picture of their behavior under a realistic request-response pattern.

The tests are conducted using the same physical setup described in Section III with the DNS server operating in the DUT. The server is configured to answer for 5 authoritative zones. The client is a custom application designed to query the server at constant, configurable request rates, with each request randomly selecting one of the available DNS zones.

Furthermore, for maximum performance, NSD is capable of leveraging multiple available CPU cores, thus also using all the available NIC's hardware queues. In all the following tests, NSD was configured to use all 6 CPU cores available.

Figure 8 depicts the average power consumption from the DUT during 10-minute runs under idle and constant request

²<https://nlnetlabs.nl/projects/nsd>

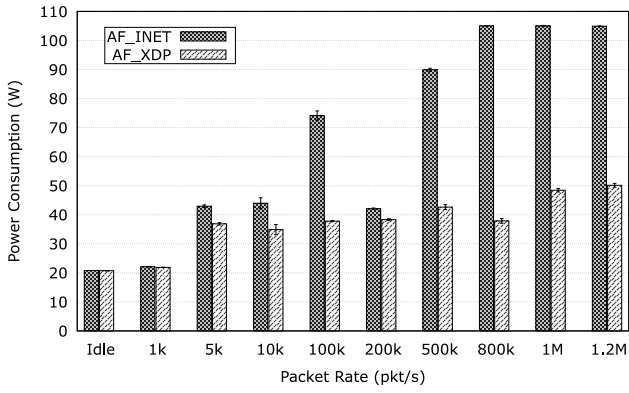


Fig. 8. Average power consumption (W)

rates scenarios. The values presented are averaged from 10 individual tests for each demand scenario and compared.

Similar to the results from the synthetic tests in Section IV, at idle, both socket families perform identically in terms of power consumption. However, as the number of requests per second increases, the difference between the two approaches becomes noticeable significantly sooner, at around 5k packets per second. At this reasonably low request rate, using XSK already results in approximately 6 W lower consumption than *AF_INET*, a 14% decrease in the overall power consumption of the DUT.

At higher request rates, energy savings increase significantly. At a rate of 500k DNS requests per second, switching from *AF_INET* sockets to *AF_XDP* allows for a drop in power consumption from 90 W to just 42 W, a staggering 53% decrease.

Takeaway #12. *The usage of XSK in the NSD allows for significant energy savings.*

Looking at the average CPU utilization, as shown in Figure 9, it is again possible to see a pattern similar to that of the synthetic tests in Section IV. Both at idle and up to around 10k requests per second, the overall CPU utilization is identical between the two socket families. With a higher request rate per second, CPU usage increases but at a much lower rate with XSK, a pattern that correlates with the power consumption.

Considering a 500k request-per-second example again, at this rate, the overall CPU utilization NSD required to meet the demand is approximately 55%. Meanwhile, leveraging XSK, NSD handles the same request rate while incurring only about 6% of overall CPU utilization, an 89% decrease.

Takeaway #13. *The usage of XSK in the NSD allows for significant reductions in CPU utilization.*

By comparing the number of requests sent by the client application against the reported number of successfully processed requests by NSD, it is possible to infer the number of packets lost. Since each DNS request over UDP consists of one single packet, the number of missing requests matches

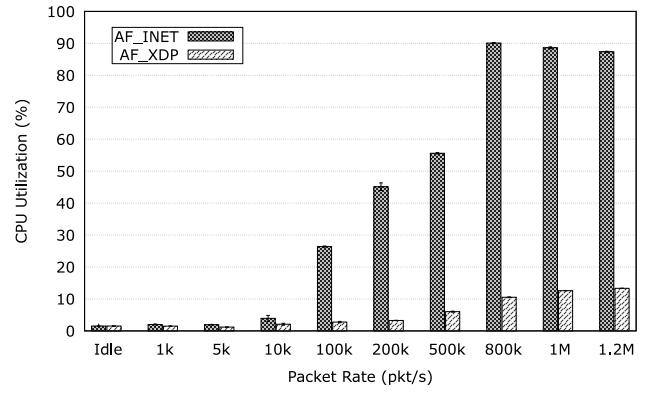


Fig. 9. Average CPU core utilization (%)

TABLE III
PACKET LOSS PERCENTAGE ACROSS *AF_INET* AND *AF_XDP* SOCKETS AT DIFFERENT REQUEST RATES.

Socket Type	1k–200k	500k	800k	1M	1.2M
	Packet Loss (%)				
<i>AF_INET</i>	-	0.06	0.72	15.82	29.54
<i>AF_XDP</i>	-	-	-	-	-

the number of lost packets.

Table III reports the observed packet loss percentages during the tests conducted to NSD while operating with either *AF_INET* or *AF_XDP* sockets. In contrast with results from the synthetic tests in Section IV, the first signs of significant packet loss occur at a lower rate of 500k requests per second.

At this rate, running NSD with *AF_INET* sockets already results in approximately 0.06% packet loss. As the number of requests per second increases, the percentage loss also increases, reaching almost 30% at 1.2M requests per second. On the other end, and in line with the results from the synthetic tests, *AF_XDP* sockets show no packet loss, enabling NSD to successfully meet the demand of all test cases.

VI. DISCUSSION

The results presented and the takeaways in Section IV allow for multiple conclusions about how to approach low-level network programming when handling UDP traffic.

As expected, blocking approaches stand out by their efficiency when compared with their non-blocking counterparts. When at idle or with reasonably low traffic demand, CPU usage is significantly lower due to blocking mechanisms that wait for an OS interrupt rather than spending numerous CPU cycles actively checking for new packets. Consequently, the power consumption of this approach is noticeably lower than that of non-blocking alternatives. As the number of packets per second increases, the gaps in power consumption and CPU usage between blocking and non-blocking approaches shrink; nevertheless, blocking strategies continue to outperform the alternatives.

Among the blocking strategies, XSK starts to stand out as network traffic demand increases during the tests. From 100k packets per second onward, *AF_XDP* sockets consistently show significantly lower CPU usage than other blocking strategies, resulting in noticeable energy savings. Furthermore, XSK is the only blocking approach that managed to run the complete test suite without packet loss, solidifying its efficiency.

On the non-blocking spectrum, the scenario changes slightly, with the three approaches no longer exhibiting equal behavior at idle and under low traffic demand. Both blocking and batch non-blocking exhibit identical performance. However, XSK non-blocking shows higher power consumption despite slightly lower CPU usage. This observation is particularly surprising, despite the more efficient packet handling of XSK, it appears that actively polling within this socket type results in a power consumption overhead.

As traffic demand increases, this overhead starts to dissipate, but only at around 800k packets per second XSK non-blocking is able to beat the remaining non-blocking approaches in terms of energy utilization. Nonetheless, after this point, the gains in power consumption remain marginal despite higher CPU usage.

Considering the most demanding test scenario, *i.e.*, 1.2M 1024-byte packets per second, only three approaches were able to surpass it with zero packet loss. Besides the two using *AF_XDP* sockets (blocking and non-blocking), standard API sockets in batch and non-blocking modes were the other options capable of handling such demand. When using standard sockets, this is, in fact, the method that yields the best performance for handling large amounts of network traffic. However, when considering both power consumption and CPU utilization, this approach clearly loses to the XSK alternatives.

Among the non-blocking solutions, the XSK version results in 7.7% lower power consumption and 28% lower CPU usage. Yet, the largest gain comes from the XSK blocking alternative. By opting for this approach instead of standard non-blocking sockets, it is possible to save up to 33.8% in power consumption and an impressive 67.5% in CPU utilization.

In this scenario, the power consumption savings amount to approximately 21 W. Alone, this value may not represent much, but consider, for example, a modest organization with 1000 similar servers that handle similar demands for just 50% of the day; after a year, the total energy saved amounts to a staggering 92000 kiloWatt hour (kWh). At an average price of 15 cents per kWh, such energy savings would amount to up to 14,000 euros.

VII. CONCLUSION

This paper highlights the superior performance of *AF_XDP* sockets compared with traditional *AF_INET* sockets. Besides the unsurprisingly better CPU utilization and lower packet drops under high network demand, this work draws attention to an unexplored aspect of the utilization of XSK: its energy efficiency.

The work demonstrates that using *AF_XDP* sockets in UDP-based applications can significantly reduce power consumption and, consequently, operating costs. Due to its efficient design, this socket family not only reduces power consumption but also sustains higher traffic demands without incurring packet loss, all while keeping CPU utilization considerably lower. These benefits were further confirmed in a real-world scenario, where enabling *AF_XDP* support in the NSD DNS server reduced the power consumption by up to 53% and the CPU utilization by up to 89% at high request rates, without any observable packet loss.

Nevertheless, the adoption of *AF_XDP* sockets is not without trade-offs. By bypassing the Kernel's network stack, functionality otherwise provided by the OS must be reimplemented at the application level, increasing the development complexity. Likewise, non-blocking configurations should be employed judiciously, since their active polling can offset part of the energy gains under low traffic demand. Even so, for UDP-based applications subject to medium and high traffic loads, *AF_XDP* sockets emerge as a compelling, more sustainable choice, capable of delivering substantial energy and cost savings without compromising QoS.

Acknowledgment This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., under the support UID/50014/2025 (<https://doi.org/10.54499/UID/50014/2025>).

REFERENCES

- [1] J. Rüth, I. Poese, C. Dietzel, and O. Hohlfeld, *A First Look at QUIC in the Wild*. Springer International Publishing, 2018, p. 255–268. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-76481-8_19
- [2] L. Belkhir and A. Elmeligi, “Assessing ICT global emissions footprint: Trends to 2040 & recommendations,” *Journal of Cleaner Production*, vol. 177, pp. 448–463, Mar. 2018.
- [3] European Commission. Joint Research Centre., *Energy Consumption in Data Centres and Broadband Communication Networks in the EU*. LU: Publications Office, 2024.
- [4] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. Câmara Júnior, and L. F. M. Vieira, “Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications,” *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–36, 2020.
- [5] eBPF Docs Contributors, “AF_XDP – eBPF docs,” https://docs.ebpf.io/linux/concepts/af_xdp/, 2024, accessed: 2026-06-15.
- [6] Marten, “Recapitulating AF_XDP,” <https://hpnpl.net/posts/recapitulating-af-xdp/>, 2024, accessed: 2026-06-15.
- [7] P. Salva-Garcia, R. Ricart-Sanchez, E. Chirivella-Perez, Q. Wang, and J. M. Alcaraz-Calero, “XDP-Based SmartNIC hardware performance acceleration for next-generation networks,” *Journal of Network and Systems Management*, vol. 30, no. 4, p. 75, 2022.
- [8] C. Delzotti, P. Maistriaux, and T. Barbette, “Sloth: A kernel-bypass scheduler maximizing energy efficiency under latency constraints,” in *IFIP Networking Conference (IFIP Networking), SLICES Workshop*, 2025.
- [9] E. Freitas, A. T. de Oliveira Filho, P. R. X. do Carmo, D. F. H. Sadok, and J. Kelner, “Takeaways from an experimental evaluation of eXpress Data Path (XDP) and Data Plane Development Kit (DPDK) under a cloud computing environment,” *Research, Society and Development*, vol. 11, no. 12, p. e26111234200, 2022.